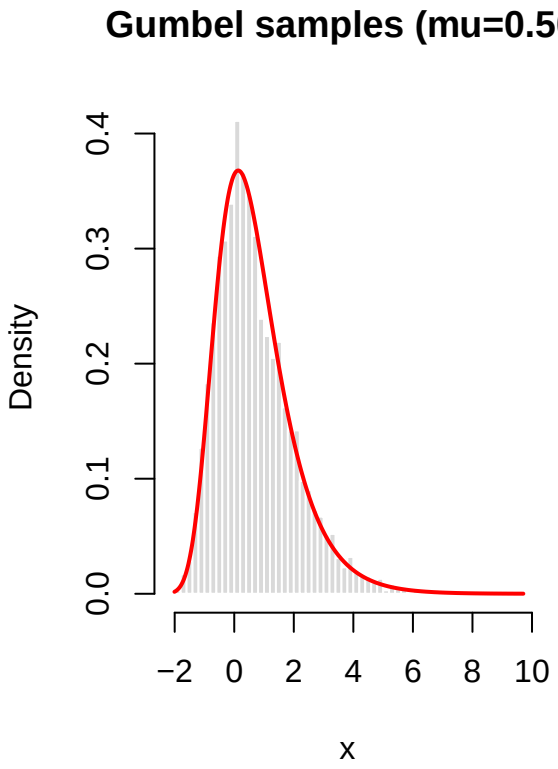
Lab 5

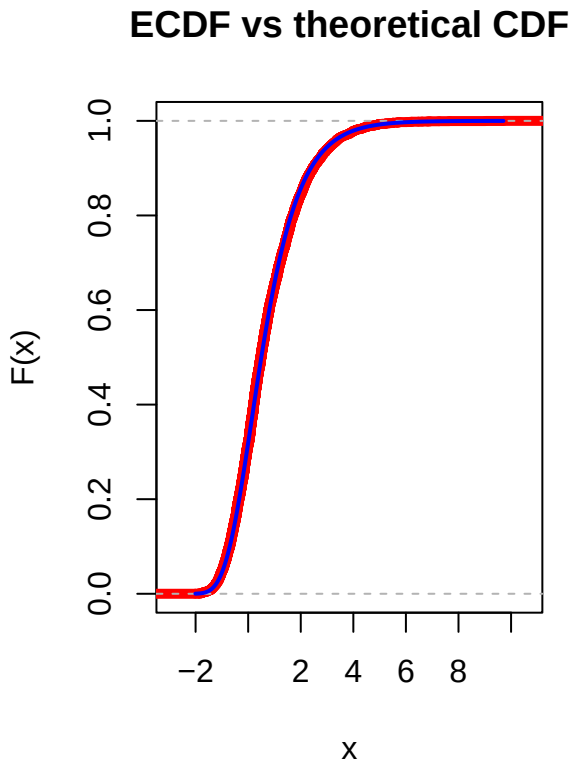
Aron Enström (aroen488), Sergey Volchkov (servo519)

2026-02-18

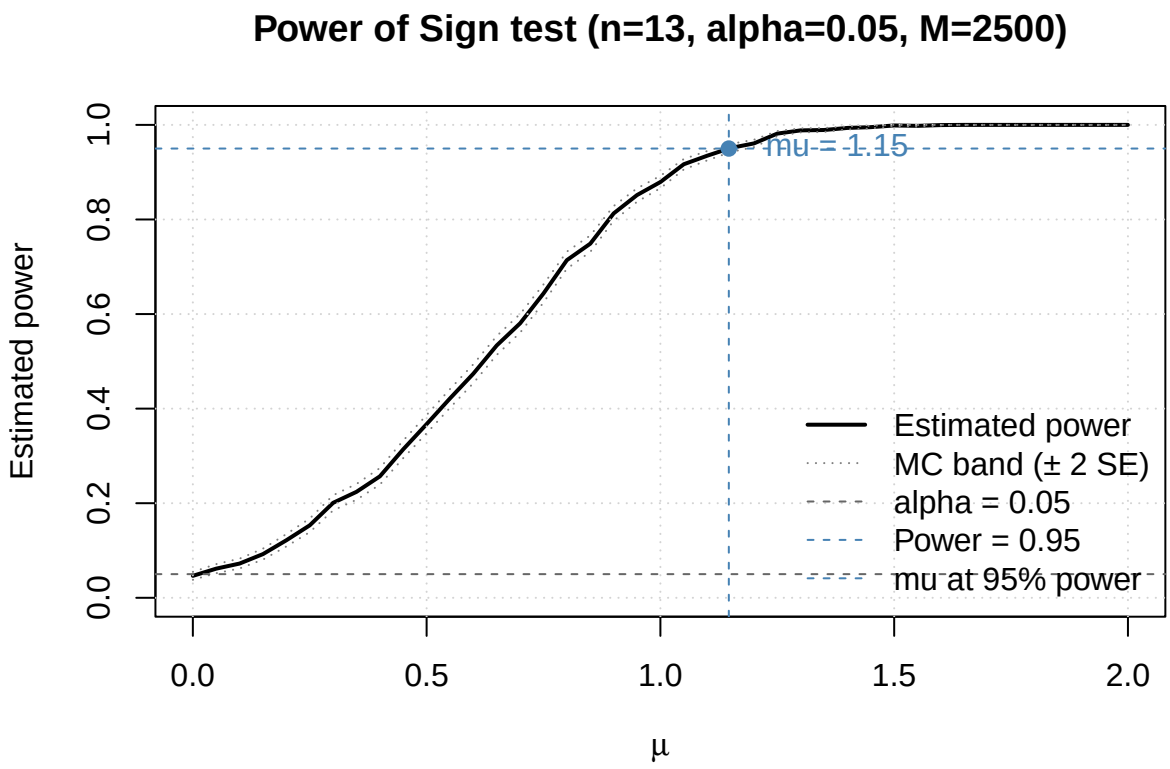
Q1:

a. Generate Gumbel random variables using the Inverse Transformation Method





b. Sign test and power curve




```
## Chosen M = 2500
```

```
## Worst-case MC SE bound = 0.01
```

The figure shows the simulated power curve of the one-sided Sign test for $H_0 : \mu = 0$ versus $H_a : \mu > 0$ with $n = 13$ at significance level $\alpha = 0.05$ (computed from $M = 2500$ Monte Carlo repetitions, which is chosen based on the desired Monte-Carlo SE of 0.01). As expected, when $\mu = 0$ the rejection probability is close to $\alpha \approx 0.05$, indicating correct Type I error control under the null. Power increases monotonically with μ, meaning the test becomes more likely to detect a positive median shift as the true median grows.

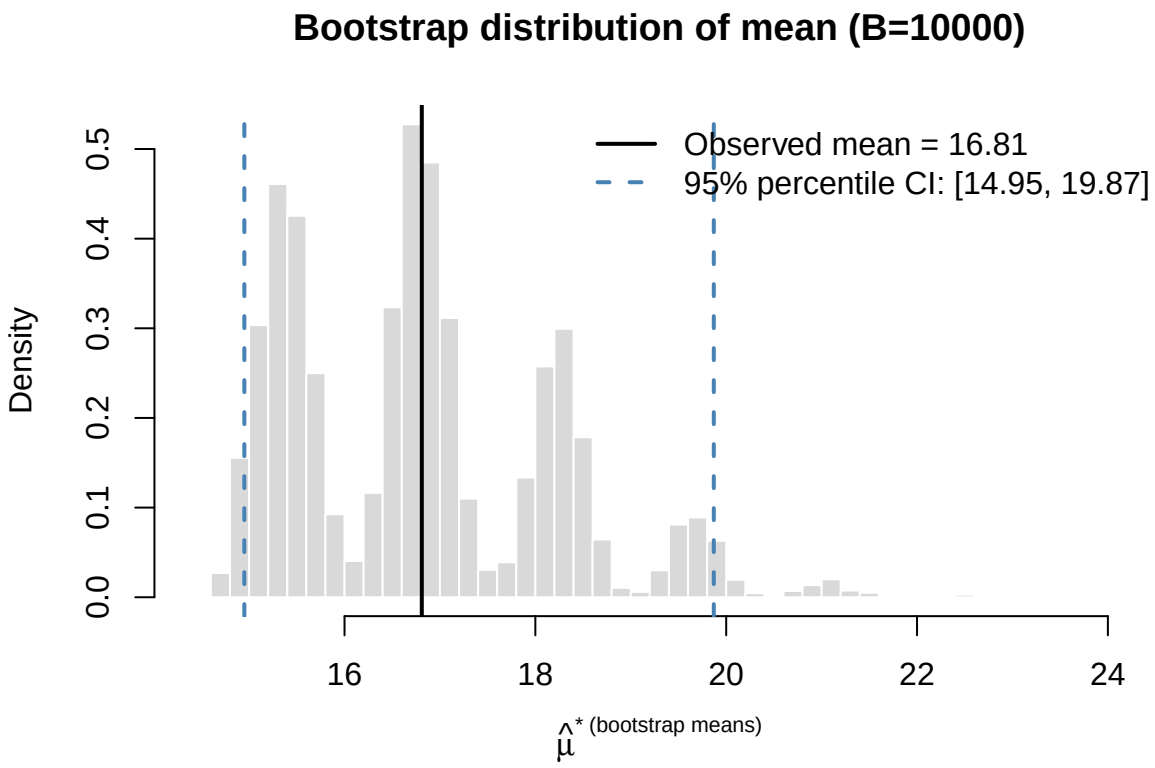
Using the horizontal reference line at power 0.95, the curve crosses 0.95 at approximately $\mu \approx 1.1463415$ (vertical dashed line), i.e., a median shift of about 1.1463415 is needed to achieve roughly 0.95 power for this setup.

Q2:

a. 95%-bootstrap CI with percentile method

```
## Sample mean = 16.81
```

```
## 95% percentile bootstrap CI for mean = [14.95, 19.87]
```



b. 95%-bootstrap CI with percentile and BCa methods and normality assumption

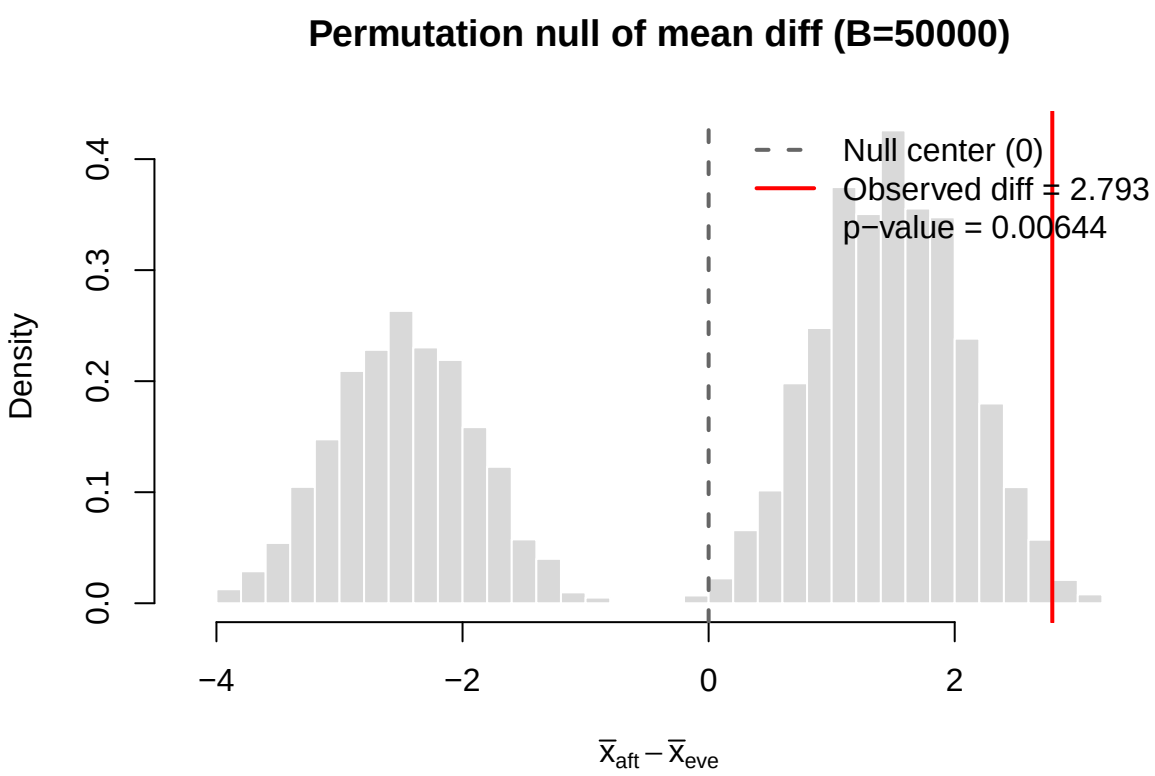
```
## Bootstrap percentile 95% CI for mean: [14.96, 19.84]
## Bootstrap BCa        95% CI for mean: [15.15, 21.42503]
## Normal-theory (t) 95% CI for mean: [13.50549, 20.11451]
```

c. Comparison

Based on n=10 observations, the estimated mean travel time is 16.81 minutes, with a 95% CI roughly spanning about 15–20 minutes (method-dependent), and the bootstrap distribution indicates right-skewness due to the 29.7-minute outlier. The BCa interval is shifted upward and has a higher upper endpoint than the percentile CI, which is typical when the bootstrap distribution is biased and/or skewed; BCa is designed to adjust for bias and skewness in the bootstrap estimates. The normal-theory CI is symmetric around the sample mean by construction, while the bootstrap distribution here is visibly asymmetric; that's why the bootstrap-based intervals (especially BCa) can be more informative when the sampling distribution is not close to symmetric.

d. Permutation test for difference in means

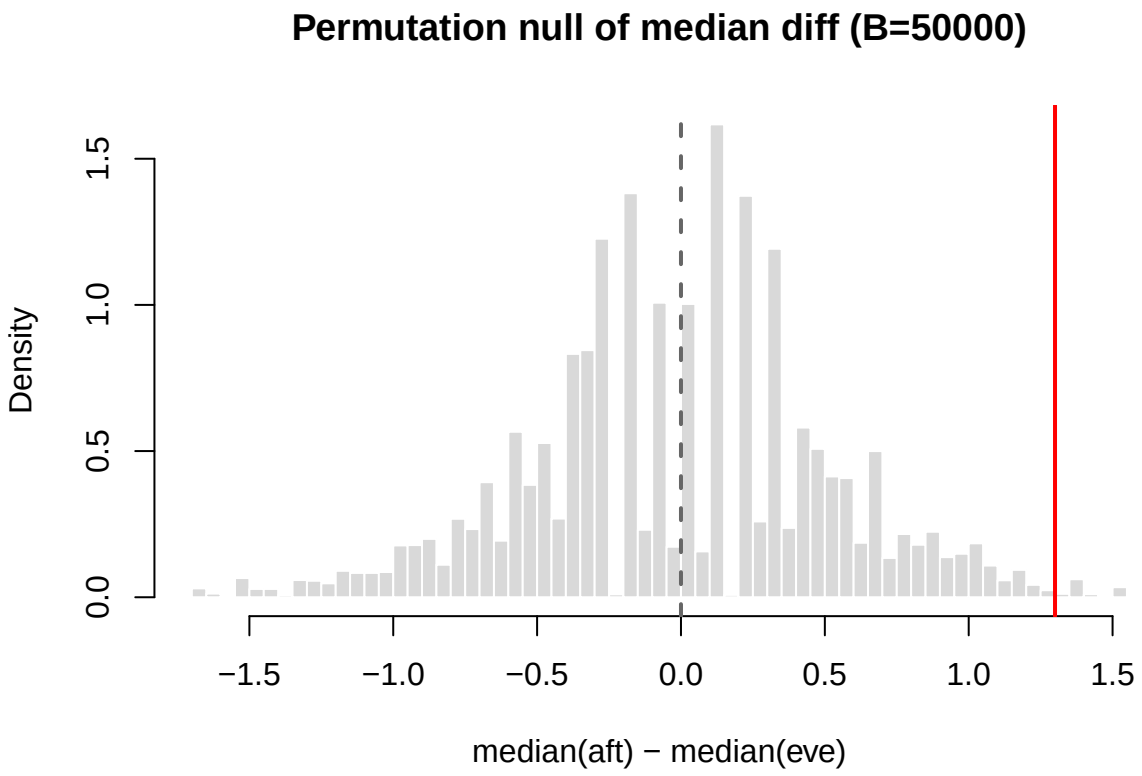
```
## Observed mean(aft) = 16.810
## Observed mean(eve) = 14.017
## Estimate mu - nu   = 2.793 (minutes)
## Permutation test p-value (one-sided, greater) = 0.00644
```



The one-sided permutation p-value is $p \approx 0.00644$, meaning that under H_0 (labels exchangeable; equal means), only about 0.6% of random relabelings produce a mean difference at least as large as the observed 2.793 minutes. Since $0.00644 < 0.05$, we can reject H_0 and conclude the afternoon mean travel time is **greater** than the evening mean (in the direction tested). However, the afternoon sample includes a large value (29.7), so the mean (and hence the mean-difference test statistic) can be sensitive to that outlier - we can see it from the two peaks in our histogram, which are due to whether the outlier was assigned to afternoon or evening times during the random permutations. A robustness check could be repeating the permutation test with a different statistic (e.g., difference in medians or trimmed means).

```
## Observed median difference (aft - eve) = 1.300
```

```
## Permutation p-value (one-sided, greater) = 0.00712
```



Based on the results of the permutation test on the median difference we can conclude that there is a statistically significant difference between the medians of the two samples. It means that afternoon trips are typically longer, and this conclusion is not being driven solely by the single extreme afternoon observation because the median is robust to outliers.

Appendix

```
#####
# 1(a)
#####

set.seed(1)

c_const <- log(log(2))    # ~ -0.3665129

rgumbel_median_mu <- function(n, mu) {
  u <- runif(n)
  x <- mu + c_const - log(-log(u))
  x
}

## Theoretical CDF
pgumbel_median_mu <- function(x, mu) {
  exp(-exp(-(x - mu - c_const)))
}
```

```r
mu_check <- 0.5
N_check  <- 5000
x <- rgumbel_median_mu(N_check, mu_check)

par(mfrow = c(1,2))

hist(x, breaks = 50, freq = FALSE, col = "grey85", border = "white",
     main = sprintf("Gumbel samples (mu=%.2f)", mu_check),
     xlab = "x")
z <- x - (mu_check + c_const)

grid <- seq(min(x), max(x), length.out = 400)
zg <- grid - (mu_check + c_const)
dg <- exp(-zg) * exp(-exp(-zg))
lines(grid, dg, col = "red", lwd = 2)

## Empirical CDF vs theoretical CDF
plot(ecdf(x), do.points = FALSE,
     main = "ECDF vs theoretical CDF", xlab = "x", ylab = "F(x)", col = "red", lwd = 5)
lines(grid, pgumbel_median_mu(grid, mu_check), col = "blue", lwd = 2)


#####
# 1(b)
#####
## H0: P(X>0)=0.5 vs Ha: P(X>0)>0.5 (corresponds to median > 0)
sign_pvalue_greater <- function(x) {
  s <- sum(x > 0)              # successes
  n <- length(x)               # trials
  binom.test(s, n, p = 0.5, alternative = "greater")$p.value
}

## Choosing repetitions M to control Monte Carlo standard error of power estimate:
## If p_hat is estimated power, Monte Carlo SE is sqrt(p_hat * (1-p_hat)/M) <= 0.5/sqrt(M).
## So, to ensure worst-case SE <= eps, we take M >= (0.5/eps)^2.
choose_M <- function(eps = 0.01) ceiling((0.5/eps)^2)

n     <- 13
alpha <- 0.05
eps   <- 0.01              # target MC SE of about 0.01 at worst case
M     <- choose_M(eps)     # e.g., eps=0.01 => M=2500

mu_grid <- seq(0, 2, by = 0.05)

## Simulate power for each mu
power_hat <- numeric(length(mu_grid))
mc_se     <- numeric(length(mu_grid))

for (j in seq_along(mu_grid)) {
  mu <- mu_grid[j]
  reject <- logical(M)

  for (m in 1:M) {
```

```
    x <- rgumbel_median_mu(n, mu)
    reject[m] <- (sign_pvalue_greater(x) <= alpha)
  }

  power_hat[j] <- mean(reject)
  mc_se[j]     <- sqrt(power_hat[j] * (1 - power_hat[j]) / M)
}

target_power <- 0.95 # 5% of Type II error, to be increased if false negatives
                     # are very costly

if (max(power_hat) < target_power) {
  mu_star <- NA_real_
  warning("Power never reaches target on this mu grid.")
} else {
  k <- which(power_hat >= target_power)[1]

  if (k == 1) {
    mu_star <- mu_grid[1]
  } else {
    mu_star <- approx(x = power_hat[(k-1):k],
                      y = mu_grid[(k-1):k],
                      xout = target_power,
                      method = "linear")$y
  }
}


plot(mu_grid, power_hat, type = "l", lwd = 2, ylim = c(0, 1),
     xlab = expression(mu), ylab = "Estimated power",
     main = sprintf("Power of Sign test (n=%d, alpha=%.2f, M=%d)", n, alpha, M))
grid()

lines(mu_grid, pmin(1, power_hat + 2*mc_se), lty = 3, col = "grey50")
lines(mu_grid, pmax(0, power_hat - 2*mc_se), lty = 3, col = "grey50")

abline(h = alpha, lty = 2, col = "grey40")

abline(h = target_power, lty = 2, col = "steelblue")
if (!is.na(mu_star)) {
  abline(v = mu_star, lty = 2, col = "steelblue")
  points(mu_star, target_power, pch = 19, col = "steelblue")
  text(mu_star, target_power,
       labels = sprintf("  mu = %.2f", mu_star),
       pos = 4, col = "steelblue")
}

legend("bottomright",
       legend = c("Estimated power", "MC band (± 2 SE)", sprintf("alpha = %.2f", alpha),
                  sprintf("Power = %.2f", target_power),
                  sprintf("mu at %.0f%% power", 100*target_power)),
       lty = c(1, 3, 2, 2, 2),
       lwd = c(2, 1, 1, 1, 1),
```

```
       col = c("black", "grey50", "grey40", "steelblue", "steelblue"),
       bty = "n")

cat("Chosen M =", M, "\n")
cat("Worst-case MC SE bound =", 0.5/sqrt(M), "\n")
#####
# 2(a)
#####
set.seed(1)

x <- c(16.5, 14.6, 15.0, 13.9, 15.4, 29.7, 14.8, 15.3, 15.9, 17.0)
n <- length(x)


B <- 10000
alpha <- 0.05

## bootstrap distribution of the mean
boot_means <- numeric(B)
for (b in 1:B) {
  xb <- sample(x, size = n, replace = TRUE)  # resample cases with replacement
  boot_means[b] <- mean(xb)
}

## empirical quantiles of bootstrap means
ci <- as.numeric(quantile(boot_means, probs = c(alpha/2, 1 - alpha/2), type = 7))

cat("Sample mean =", mean(x), "\n")
cat("95% percentile bootstrap CI for mean = [", ci[1], ", ", ci[2], "]\n", sep = "")

hist(boot_means, breaks = 40, freq = FALSE, col = "grey85", border = "white",
     main = sprintf("Bootstrap distribution of mean (B=%d)", B),
     xlab = expression(hat(mu)^"* (bootstrap means)"))

abline(v = mean(x), col = "black", lwd = 2)
abline(v = ci[1], col = "steelblue", lwd = 2, lty = 2)
abline(v = ci[2], col = "steelblue", lwd = 2, lty = 2)

legend("topright",
       legend = c(sprintf("Observed mean = %.2f", mean(x)),
                  sprintf("95%% percentile CI: [%.2f, %.2f]", ci[1], ci[2])),
       lwd = c(2, 2), lty = c(1, 2), col = c("black", "steelblue"),
       bty = "n")

#####
# 2(b)
#####
library(boot)

boot_mean <- function(d, i) mean(d[i])

set.seed(1)
B <- 10000
```

```r
boot_out <- boot(data = x, statistic = boot_mean, R = B)

## 95% percentile CI
ci_perc <- boot.ci(boot_out, conf = 0.95, type = "perc")
ci_perc_95 <- ci_perc$percent[4:5]

## 95% BCa CI
ci_bca <- boot.ci(boot_out, conf = 0.95, type = "bca")
ci_bca_95 <- ci_bca$bca[4:5]

cat("Bootstrap percentile 95% CI for mean: [", ci_perc_95[1], ", ", ci_perc_95[2], "]\n", sep="")
cat("Bootstrap BCa        95% CI for mean: [", ci_bca_95[1],  ", ", ci_bca_95[2],  "]\n", sep="")

## 95% CI assuming Normal model for the data
n <- length(x)
xbar <- mean(x)
s <- sd(x)
tcrit <- qt(0.975, df = n - 1)
ci_norm_95 <- xbar + c(-1, 1) * tcrit * s / sqrt(n)

cat("Normal-theory (t) 95% CI for mean: [", ci_norm_95[1], ", ", ci_norm_95[2], "]\n", sep="")
#####
# 2(d)
#####

set.seed(1)

aft <- x
eve <- c(15.0, 13.0, 12.9, 14.4, 15.1, 13.7)

d_obs <- mean(aft) - mean(eve)

cat(sprintf("Observed mean(aft) = %.3f\n", mean(aft)))
cat(sprintf("Observed mean(eve) = %.3f\n", mean(eve)))
cat(sprintf("Estimate mu - nu   = %.3f (minutes)\n\n", d_obs))

B <- 50000
n1 <- length(aft)
n2 <- length(eve)

pool <- c(aft, eve)

d_perm <- numeric(B)
for (b in 1:B) {
  perm <- sample(pool, size = n1 + n2, replace = FALSE)
  g1 <- perm[1:n1]
  g2 <- perm[(n1 + 1):(n1 + n2)]
  d_perm[b] <- mean(g1) - mean(g2)
}

p_val <- (sum(d_perm >= d_obs) + 1) / (B + 1)

cat(sprintf("Permutation test p-value (one-sided, greater) = %.5f\n", p_val))
```

```r
hist(d_perm, breaks = 50, freq = FALSE, col = "grey85", border = "white",
     main = sprintf("Permutation null of mean diff (B=%d)", B),
     xlab = expression(bar(x)[aft] - bar(x)[eve]))

abline(v = 0, col = "grey40", lty = 2, lwd = 2)
abline(v = d_obs, col = "red", lwd = 2)

legend("topright",
       legend = c("Null center (0)", sprintf("Observed diff = %.3f", d_obs),
                  sprintf("p-value = %.5f", p_val)),
       lty = c(2, 1, NA), lwd = c(2, 2, NA),
       col = c("grey40", "red", "black"),
       bty = "n")
stat_obs <- median(aft) - median(eve)

B <- 50000
n1 <- length(aft); n2 <- length(eve)
pool <- c(aft, eve)

stat_perm <- numeric(B)
for (b in 1:B) {
  perm <- sample(pool, length(pool), replace = FALSE)
  g1 <- perm[1:n1]
  g2 <- perm[(n1+1):(n1+n2)]
  stat_perm[b] <- median(g1) - median(g2)
}

p_val <- (sum(stat_perm >= stat_obs) + 1) / (B + 1)

cat(sprintf("Observed median difference (aft - eve) = %.3f\n", stat_obs))
cat(sprintf("Permutation p-value (one-sided, greater) = %.5f\n", p_val))

hist(stat_perm, breaks = 50, freq = FALSE, col = "grey85", border = "white",
     main = sprintf("Permutation null of median diff (B=%d)", B),
     xlab = "median(aft) - median(eve)")
abline(v = 0, col = "grey40", lty = 2, lwd = 2)
abline(v = stat_obs, col = "red", lwd = 2)
```